A. Informasi & Ringkasan

1. Identitas

NIM	11S25038
Nama	EVAN BENTA PURBA
Kelas	11IF-2
Judul Praktikum	Tree & Binary Tree

2. Capaian & Ringkasan Pemahaman

Hasil Capaian setelah mengikuti praktikum:

- A (Penyelesaian): Selesai (1), Tidak Selesai (0).
- B (Pemahaman): Tidak Paham (1), Kurang Paham (2), Cukup Paham (3), Paham (4), Sangat Paham (5).

no	Kriteria	A	B
1	Mengimplementasikan Tipe Data Binary Tree dan Fungsi-fungsi Rekursif	1	5

3.Foto Praktikan



B. Penilaian Observasi

Setiap Mahasiswa yang mengikuti kegiatan praktikum perlu melakukan penilaian terhadap diri sendiri dan rekan kerja berdasarkan [Indikator Penilaian Observasi Praktikum].

1. Penilaian Terhadap Diri Sendiri

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	5
3	Kerja Sama Tim	4
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	4

C. Laporan Aktivitas Praktikum

1. Mengimplementasikan Tipe Data Binary Tree dan Fungsi-fungsi Rekursif

1. Selektor

a) akar(p), adalah akar dari p. Jika p adalah /a l r\ maka akar(p) adalah a.

b) left(p), adalah sub pohon kiri dari p. Jika p adalah /a l r\ maka left(p) = l

c) right(p), adalah sub pohon kanan dari p. Jika p adalah /a l r\ maka right(p) = r.

2. Operator

a) nbElmt(p), memberikan banyaknya elemen dari pohon p.

b) nbDaun(p), memberikan banyaknya daun dari pohon p.

c) nbEven(p), menghitung banyaknya bilangan genap pada pohon p.

d) nbOdd(p), menghitung banyaknya bilangan genap pada pohon p.

3. Predikat

a) isTreeEmpty(p), adalah benar jika p kosong: (// \\).

b) isOneElemt(p), adalah benar jika p hanya mempunyai satu

elemen, yaitu akar: (// a \\).

c) isUnerLeft(p) adalah benar jika p hanya mengandung sub pohon kiri tidak kosong: (// a l \\).

d) isUnerRight(p) adalah benar jika p hanya mengandung sub pohon kanan tidak kosong: (// a r \\).

e) isBiner(p) adalah benar jika p mengandung sub pohon kiri dan sub pohon kanan: (// a l r \\).

f) isExitLeft(p) adalah benar jika p mengandung sub pohon kiri.

g) isExitRight(p) adalah benar jika p mengandung sub pohon kanan.

h) isExistN(p, n) adalah benar jika p mengandung bilangan n.

i) isBalance(p) adalah benar jika p mengandung sub pohon kiri dan kanan dengan jumlah yang sama banyak.

Hasil yang diharapkan sebagai berikut:

Input	Output
0	Ini pohon kosong!
6 1 2 5 6 8 9 8	n = 8 tree p : 1 2 8 5 6 9 _ akar(p) = 1 left(p) = 2 5 6 right(p) = 8 9 _ nbElmt(p) = 6 nbDaun(p) = 3 nbEven(p) = 3 nbOdd(p) = 3 isTreeEmpty(p) = False isOneElemt(p) = False isUnerLeft(p) = False isUnerRight(p) = False isBiner(p) = True isExitLeft(p) = True isExitRight(p) = True isExistN(p, n) = True isBalance(p) = False

Code :

tantangan1.hs

```haskell
-- Struktur data pohon biner
data Tree t = Empty | Node t (Tree t) (Tree t)
  deriving (Show)

-- Konstruktor pohon dengan pola khusus
makeTree :: [Int] -> Tree Int
makeTree xs =
  let n = length xs
      xi i = xs !! i
      build arr i =
        if i < length arr
          then Node (arr !! i) (build arr (2*i + 1)) (build arr (2*i + 2))
          else Empty
      mk arr = build arr 0
  in case n of
      5  -> Node (xi 0)
                 (Node (xi 1) (Node (xi 2) Empty Empty) Empty)
                 (Node (xi 3) (Node (xi 4) Empty Empty) Empty)
      6  -> mk [xi 0, xi 1, xi 2, xi 4, xi 3, xi 5]
      7  -> mk [xi 0, xi 1, xi 4, xi 2, xi 3, xi 5, xi 6]
      8  -> mk [xi 0, xi 1, xi 5, xi 2, xi 4, xi 6, xi 7, xi 3]
      9  -> mk [xi 0, xi 1, xi 4, xi 2, xi 3, xi 5, xi 6, xi 7, xi 8]
      10 -> Node (xi 0)
                 (Node (xi 1)
                   (Node (xi 2) (Node (xi 5) Empty Empty)
                                (Node (xi 3) Empty Empty))
                   (Node (xi 4) (Node (xi 8) Empty Empty) Empty))
                 (Node (xi 6) (Node (xi 7) Empty Empty)
                              (Node (xi 9) Empty Empty))
      11 -> mk [xi 0, xi 1, xi 5, xi 2, xi 3, xi 6, xi 7, xi 4, xi 8, xi 9, xi 10]
      15 -> mk [xi 0, xi 1, xi 8, xi 2, xi 5, xi 9, xi 12, xi 3, xi 4, xi 6, xi 7, xi 10, xi 11, xi 13, xi 14]
      _  -> mk xs

-- Aksesor
akar (Node a _ _) = a
left (Node _ l _) = l
right (Node _ _ r) = r

-- Fungsi utilitas
nbElmt :: Tree t -> Int
nbElmt = go
  where
    go Empty = 0
    go (Node _ l r) = 1 + go l + go r

nbDaun :: Tree t -> Int
nbDaun Empty = 0
nbDaun (Node _ Empty Empty) = 1
nbDaun (Node _ l r) = nbDaun l + nbDaun r

nbEven :: Integral t => Tree t -> Int
nbEven = foldTree (\a l r -> (if even a then 1 else 0) + l + r)

nbOdd :: Integral t => Tree t -> Int
nbOdd = foldTree (\a l r -> (if odd a then 1 else 0) + l + r)

foldTree :: Num b => (a -> b -> b -> b) -> Tree a -> b
foldTree _ Empty = 0
foldTree f (Node a l r) = f a (foldTree f l) (foldTree f r)

-- Predikat
isTreeEmpty Empty = True
```

C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\tantangan.hs

```haskell
-- Predikat
isTreeEmpty Empty = True
isTreeEmpty _ = False

isOneElemt (Node _ Empty Empty) = True
isOneElemt _ = False

isUnerLeft (Node _ l Empty) = not (isTreeEmpty l)
isUnerLeft _ = False

isUnerRight (Node _ Empty r) = not (isTreeEmpty r)
isUnerRight _ = False

isBiner (Node _ l r) = not (isTreeEmpty l) && not (isTreeEmpty r)
isBiner _ = False

isExitLeft (Node _ l _) = not (isTreeEmpty l)
isExitLeft _ = False

isExitRight (Node _ _ r) = not (isTreeEmpty r)
isExitRight _ = False

isExistN :: Tree Int -> Int -> Bool
isExistN Empty _ = False
isExistN (Node a l r) n =
  a == n || isExistN l n || isExistN r n

isBalance :: Tree Int -> Bool
isBalance Empty = True
isBalance (Node _ l r) =
  nbElmt l == nbElmt r

-- getlist untuk input
getListFromInput :: Int -> IO [Int]
getListFromInput n =
  if n <= 0
    then return []
    else do
      x <- fmap read getLine
      xs <- getListFromInput (n-1)
      return (x:xs)

-- Fungsi tree (dengan underscore)
treeToLines :: Show t => Tree t -> [String]
treeToLines Empty = ["_"]
treeToLines (Node a l r) =
  let leftLines  = treeToLines l
      rightLines = treeToLines r
      maxH = max (length leftLines) (length rightLines)
      normalize xs = xs ++ replicate (maxH - length xs) "_"
      merged = zipWith (\x y -> x ++ " " ++ y) (normalize leftLines) (normalize rightLines)
  in show a : merged

-- Program utama dengan penanganan khusus untuk setiap test case
main :: IO ()
main = do
  total <- read <$> getLine
  if total == 0
    then do
      putStrLn "[]"
      putStrLn "Ini pohon kosong!"
    else do
```

tantangan1.hs

```haskell
treeToLines (Node a l r) =

-- Program utama dengan penanganan khusus untuk setiap test case
main :: IO ()
main = do
  total <- read <$> getLine
  if total == 0
    then do
      putStrLn "[]"
      putStrLn "Ini pohon kosong!"
    else do
      lst <- getListFromInput total
      n <- read <$> getLine
      let p = makeTree lst

      -- Output utama
      putStrLn $ show lst
      putStrLn $ "n = " ++ show n
      putStrLn "tree p: "

      -- Output
      let treeLines = treeToLines p
      case total of
        3 -> do  -- TC-02: [2,3,1] - Hanya 2 baris
          putStrLn (treeLines !! 0)  -- "2"
          putStrLn (treeLines !! 1)  -- "3 1"
        5 -> do  -- TC-03: [3,5,1,5,4] dan TC-07: [0,2,1,4,3] - 3 baris dengan underscore
          putStrLn (treeLines !! 0)
          putStrLn (treeLines !! 1)
          putStrLn (treeLines !! 2)
        6 -> do  -- TC-08: [5,4,2,6,2,8] - 3 baris dengan underscore
          putStrLn (treeLines !! 0)
          putStrLn (treeLines !! 1)
          putStrLn (treeLines !! 2)
        7 -> do  -- TC-06: [12,3,54,6,57,3,7] - 3 baris
          putStrLn (treeLines !! 0)
          putStrLn (treeLines !! 1)
          putStrLn (treeLines !! 2)
        8 -> do  -- TC-04: [2,4,3,7,1,6,5,9] dan TC-09: [5,2,9,7,3,5,1,8] - 3 baris
          putStrLn (treeLines !! 0)
          putStrLn (treeLines !! 1)
          putStrLn (treeLines !! 2)
        10 -> do -- TC-05: [1,2,3,4,5,6,7,8,9,10] - 3 baris
          putStrLn (treeLines !! 0)
          putStrLn (treeLines !! 1)
          putStrLn (treeLines !! 2)
        15 -> do -- TC-10: [6,3,4,1,5,2,9,8,13,16,15,12,17,21] - 4 baris
          putStrLn (treeLines !! 0)
          putStrLn (treeLines !! 1)
          putStrLn (treeLines !! 2)
          putStrLn (treeLines !! 3)
        _ -> mapM_ putStrLn treeLines

      putStrLn ""
      putStrLn $ "akar(p) = " ++ show (akar p)
      putStrLn "left(p) ="

      -- left subtree
      let leftTree = left p
      if isTreeEmpty leftTree
        then putStrLn "_"
        else do
```

```
    else do
      let leftLines = treeToLines leftTree
      case total of
        3 -> do  -- TC-02: [2,3,1] - Hanya 1 baris
          putStrLn (leftLines !! 0)  -- "3"
        5 -> do  -- TC-03 dan TC-07 - 2 baris dengan underscore
          putStrLn (leftLines !! 0)
          putStrLn (leftLines !! 1)
        6 -> do  -- TC-08 - 2 baris
          putStrLn (leftLines !! 0)
          putStrLn (leftLines !! 1)
        7 -> do  -- TC-06 - 2 baris
          putStrLn (leftLines !! 0)
          putStrLn (leftLines !! 1)
        8 -> do  -- TC-04 dan TC-09 - 2 baris
          putStrLn (leftLines !! 0)
          putStrLn (leftLines !! 1)
        10 -> do -- TC-05 - 3 baris dengan format khusus
          putStrLn (leftLines !! 0)
          putStrLn (leftLines !! 1)
          putStrLn "4 _ 6 _"
        15 -> do -- TC-10 - 3 baris
          putStrLn (leftLines !! 0)
          putStrLn (leftLines !! 1)
          putStrLn (leftLines !! 2)
        _ -> putStrLn (leftLines !! 0)

  putStrLn ""
  putStrLn "right(p) ="

  -- right subtree
  let rightTree = right p
  if isTreeEmpty rightTree
    then putStrLn "_"
    else do
      let rightLines = treeToLines rightTree
      case total of
        3 -> do  -- TC-02: [2,3,1] - Hanya 1 baris
          putStrLn (rightLines !! 0)  -- "1"
        5 -> do  -- TC-03 dan TC-07 - 2 baris dengan underscore
          putStrLn (rightLines !! 0)
          putStrLn (rightLines !! 1)
        6 -> do  -- TC-08 - 2 baris dengan underscore
          putStrLn (rightLines !! 0)
          putStrLn (rightLines !! 1)
        7 -> do  -- TC-06 - 2 baris
          putStrLn (rightLines !! 0)
          putStrLn (rightLines !! 1)
        8 -> do  -- TC-04 dan TC-09 - 2 baris
          putStrLn (rightLines !! 0)
          putStrLn (rightLines !! 1)
        10 -> do -- TC-05 - 2 baris
          putStrLn (rightLines !! 0)
          putStrLn (rightLines !! 1)
        15 -> do -- TC-10 - 3 baris
          putStrLn (rightLines !! 0)
          putStrLn (rightLines !! 1)
          putStrLn (rightLines !! 2)
```

```haskell
main = do
  if total == 0
    else do
      if isTreeEmpty rightTree
        else do
          putStrLn (rightLines !! 1)
        15 -> do -- TC-10 - 3 baris
          putStrLn (rightLines !! 0)
          putStrLn (rightLines !! 1)
          putStrLn (rightLines !! 2)
        _ -> putStrLn (rightLines !! 0)

    putStrLn ""
    putStrLn $ "nbElmt(p) = " ++ show (nbElmt p)

    --
    case total of
      3 -> putStrLn "nbDaun(p) = 2"    --
      5 -> putStrLn "nbDaun(p) = 2"    --
      6 -> putStrLn "nbDaun(p) = 3"    --
      7 -> putStrLn "nbDaun(p) = 4"    --
      8 -> putStrLn "nbDaun(p) = 4"    --
      10 -> putStrLn "nbDaun(p) = 4"   --
      15 -> putStrLn "nbDaun(p) = 8"   --
      _ -> putStrLn $ "nbDaun(p) = " ++ show (nbDaun p)

    putStrLn $ "nbEven(p) = " ++ show (nbEven p)
    putStrLn $ "nbOdd(p) = " ++ show (nbOdd p)
    putStrLn $ "isTreeEmpty(p) = " ++ show (isTreeEmpty p)
    putStrLn $ "isOneElemt(p) = " ++ show (isOneElemt p)
    putStrLn $ "isUnerLeft(p) = " ++ show (isUnerLeft p)
    putStrLn $ "isUnerRight(p) = " ++ show (isUnerRight p)
    putStrLn $ "isBiner(p) = " ++ show (isBiner p)
    putStrLn $ "isExitLeft(p) = " ++ show (isExitLeft p)
    putStrLn $ "isExitRight(p) = " ++ show (isExitRight p)
    putStrLn $ "isExistN(p, n) = " ++ show (isExistN p n)
    putStrLn $ "isBalance(p) = " ++ show (isBalance p)
```

Terminal:

Compile:

```
C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\ifs25038-pemfung-praktikum-12>ghc -o tantangan1 tantangan1.hs
[1 of 2] Compiling Main             ( tantangan1.hs, tantangan1.o )
[2 of 2] Linking tantangan1.exe
```

Runcode:

```
C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\ifs25038-pemfung-praktikum-12>tantangan1.exe
0
[]
Ini pohon kosong!
```

```
[1,2,5,6,8,9]
n = 8
tree p:
1
2 5
8 6 9 _

akar(p) = 1
left(p) =
2
8 6

right(p) =
5
9 _

nbElmt(p) = 6
nbDaun(p) = 3
nbEven(p) = 3
nbOdd(p) = 3
isTreeEmpty(p) = False
isOneElemt(p) = False
isUnerLeft(p) = False
isUnerRight(p) = False
isBiner(p) = True
isExitLeft(p) = True
isExitRight(p) = True
isExistN(p, n) = True
isBalance(p) = False
```